

Examen réparti 1 Master 1 SAR UE MU4IN401

L. Arantes, P. Sens, J. Sopena

Novembre 2021

2 heures – **Documents non autorisés**

Tout appareil électronique interdit

Le barème est donné à titre indicatif

Exercice 1 – Callout (4 points)

On considère l'Unix du TD avec la table globale callout de type struct *callo qui gère l'ensemble des callout.

```
struct callo
{
    int      c_time;           /*      incremental time      */
    caddr_t  c_arg;           /*      argument to routine   */
    int      (*c_func)();     /*      routine                */
};
```

Question 1 1½ points

Initialement, la table ne contient aucune fonction. Donnez l'état de la table suite aux 3 appels fait dans cet ordre :

```
timeout(func1, 0, 20);
timeout(func2, 0, 20);
timeout(func3, 0, 25);
```

12 18 0 5
17 17 0 5
22 17 0 4 1

Question 2 1½ points

Deux ticks après les 3 premiers appels à timeout, on fait l'appel suivant :

```
timeout(func4, 0, 21);
```

Donnez l'état de la table après cet appel.

17 5

Question 3 1 point

En supposant que le traitement de l'interruption horloge soit terminé, donnez l'état de la table 22 ticks après le timeout de la Question 2.

Exercice 2 – Appel système (3 points)

On souhaite écrire un nouvel appel système `int nb_fathers()`. Cet appel retourne le nombre de processus père, i.e., le nombre de processus qui ont au moins un processus fils qui ne s'est pas terminé.

Question 1 3 points

Donnez le code dans le noyau de l'appel système `sys_nb_fathers()` correspondant à `int nb_fathers()`.

Exercice 3 – Signaux (5,5 points)

Question 1 $\frac{1}{2}$ point

Quel champ de la structure `proc` stocke les signaux pendants (i.e., non encore traités) d'un processus ? Pourquoi ce champ appartient-il à la structure `proc` et non à la structure `user` ?

Question 2 $\frac{1}{2}$ point

Supposons qu'un processus P1 envoie (appel à kill) un signal à un processus P2 qui a défini un handler pour ce signal. En supposant qu'il n'y ait pas d'autres signaux émis, quand le code de ce handler s'exécutera-t-il et dans quel mode (utilisateur ou système) ?

Question 3 $\frac{1}{2}$ point

Dans ce noyau (qui n'a pas de signaux masqués), un processus élu s'exécutant en mode U peut-il avoir un signal pendant ? Justifiez votre réponse.

Question 4 2 points

Nous voulons définir l'appel système `int pause ( int s)` qui retourne 0 si le signal `s` est pendant et sinon bloque l'appelant jusqu'à la réception d'un signal quelconque. Donnez le code dans le noyau de l'appel système `sys_pause()` correspondant à `int pause( int s)`. Le processus doit s'endormir avec la priorité PRISIG (interruptible par des signaux donc supérieure à PZERO) sur l'adresse de la variable du noyau qui sauvegarde les signaux pendants du processus appelant.

Question 5 $\frac{1}{2}$ point

Quelle fonction réveillera un processus endormi dans `sys_pause()` ? Que se passerait-il si PRISIG était non interruptible par des signaux ?

Question 6 $1\frac{1}{2}$ points

Nous voulons maintenant offrir l'appel système `int nbsig ()` qui retourne le nombre de signaux pendants du processus courants. Donnez le code dans le noyau de l'appel système `sys_nbsig()` correspondant à `int nbsig()`.

Exercice 4 – Synchronisation (7,5 points)

On veut créer l'appel système `int max_proc_srun( int max_srun)` qui bloque le processus appelant tant qu'il y a strictement plus de `max_srun` processus dans l'état SRUN. Pour cela, on ajoute un champs `int p_maxsrun` dans la structure `proc`. A la création d'un processus `p_maxsrun` est initialisé à 0. En cas de blocage dans `max_proc_run`, le processus se bloquera à une priorité PWRUN non interruptible. L'appel `max_proc_srun` retourne 0.

Pour réveiller les processus bloqués par `max_proc_run`, la fonction interne `verify_nb_srun` est appelée par le noyau lorsque le nombre de processus SRUN diminue.

Question 1 $\frac{1}{2}$ point

Quelles fonctions du noyau doivent appeler `verify_nb_srun` ?

Question 2 $1\frac{1}{2}$ points

Dans les questions suivantes nous aurons besoin d'une fonction interne du noyau qui retourne le nombre de processus à l'état SRUN. Donnez le code d'une telle fonction.

Question 3 $\frac{1}{2}$ point

Est-il possible que lorsqu'un processus bloqué dans `max_proc_run` est réveillé, le nombre de processus SRUN courant ne soit plus inférieur à `max_srun`? Justifiez votre réponse.

Question 4 $2\frac{1}{2}$ points

Donnez le code dans le noyau de l'appel système `sys_max_proc_srun()` correspondant à `int max_proc_srun(int max_srun)`.

Question 5 $2\frac{1}{2}$ points

Donnez le code de la fonction interne `void verify_nb_srun()`.

```

struct proc
{
    short    p_addr;        /* address of swappable image */
    short    p_size;        /* size of swappable image ( in blocks ) */
    int      p_flag;        /* process flags */
    char     p_stat;        /* process state */
    char     p_pri;        /* priority , negative is high */
    char     p_nice;        /* nice for scheduling */
    long     p_sig;        /* signal number sent to this process */
    short    p_uid;        /* real user id , used to direct tty signals */
    short    p_suid;        /* set ( effective ) user id */
    short    p_time;        /* resident time for scheduling */
    int      p_cpu;        /* cpu usage for scheduling */
    short    *p_ttyp;        /* controlling tty */
    short    p_pid;        /* unique process id */
    short    p_ppid;        /* process id of parent */
    caddr_t  p_wchan;        /* event process is awaiting */
    struct text *p_textp;    /* pointer to text structure */
    short    p_tsize;        /* size of text */
    short    p_ssize;        /* size of stack */
    short    p_clktim;        /* time to alarm clock signal */
} proc[NPROC];

/* stat codes */
#define SSLEEP 1    /* awaiting an event */
#define SRUN 3    /* running */
#define SZOMB 5    /* intermediate state in process termination */

struct user
{
    int      u_rsav[2];        /* saved env . for process switching */
    int      u_ssav[2];        /* saved env . for swapping */
    int      u_qsav[2];        /* saved env . for signaling */
    struct proc *u_procp;    /* pointer to proc structure */
    char     u_error;        /* return error code */
    char     u_intflg;        /* catch intr from sys */
    int      *u_ar0;        /* address of users saved R0 */
    int      u_arg[20];        /* arguments to current system call */
    int      *u_ap;        /* pointer to arglist */
    struct file *u_ofile[NOFILE]; /* pointers to open file */
    int      u_signal[NSIG];    /* disposition of signals */
    int      u_uid;        /* effective user id */
    int      u_gid;        /* effective group id */
    int      u_ruid;        /* real user id */
    int      u_rgid;        /* real group id */
    int      u_tsize;        /* text size ( in blocs ) */
    int      u_dsize;        /* data size ( in blocs ) */
    int      u_ssize;        /* stack size ( in blocs ) */
    int      u_csize;        /* amount of stack in use ( in blocs */

    long     u_utime;        /* this process user time */
    long     u_stime;        /* this process system time */
    long     u_cutime;        /* sum of childs ' utimes */
    long     u_cstime;        /* sum of childs ' stimes */
    ...
} u;

```

```

/*
 * kill system call
 */
kill()
{
    register struct proc *p, *q;
    register a;
    int f;

    f = 0;
    a = u.u_arg[1];
    q = u.u_procp;
    for(p = &proc[0]; p < &proc[NPROC]; p++) {
        if(p->p_stat == NULL)
            continue;
        if(a != 0 && p->p_pid != a)
            continue;
        if(a == 0 && (p->p_ttyp != q->p_ttyp || p <= &proc[1]))
            continue;
        if(u.u_uid != 0 && u.u_uid != p->p_uid)
            continue;
        f++;
        psignal(p, u.u_arg[0]);
    }
    if(f == 0)
        u.u_error = ESRCH;
}

```

```

/*
 * Send the specified signal to
 * the specified process .
 */
psignal(p, sig)
register struct proc *p;
register sig;
{
    if((unsigned)sig >= NSIG)
        return;
    if(sig) {
        p->p_sig |= 1<<(sig-1);
        if(p->p_stat == SSLEEP && p->p_pri > PZERO)
            setrun(p);
    }
}

```

```

/*
 * find the signal in bit - position
 * representation in p_sig .
 */
fsig(p)
struct proc *p;
{
    register n, i;

    n = p->p_sig;
    for(i=1; i<NSIG; i++) {
        if(n & 1)
            return(i);
    }
}

```



```

        n >>= 1;
    }
    return(0);
}

sleep(chan, pri)
caddr_t chan;
{
    register struct  proc *rp = u.u_procp;
    register s, op;

    if (pri > PZERO) {
        if(issig())
            goto psig;
        spl(CLINHB);
        rp->p_wchan = chan;
        rp->p_stat = SSLEEP;
        rp->p_pri = pri;
        spl(NORMAL);
        if(runin != 0) {
            runin = 0;
            wakeup(&runin);
        }
        swtch();
        if(issig())
            goto psig;
    } else {
        spl(CLINHB);
        rp->p_wchan = chan;
        rp->p_stat = SSLEEP;
        rp->p_pri = pri;
        spl(NORMAL);
        swtch();
    }
    return;

    /*
     * If priority was low (> PZERO) and
     * there has been a signal ,
     * execute non-local goto to
     * the qsav location .
     */
    psig:
    aretu(u.u_qsav);
}

```